Stream API

MAP – Cursul 5 · Java 8+ · pachetul `streams`

Ce este un stream

- O secvență de elemente dintr-o **sursă** (colecție, tablou, fișier), pe care se aplică operații în lanț.
- **Nu stochează** date și **nu modifică sursa**.
- Este **leneș**: nu se execută nimic până la operația terminală.
- Se parcurge **o singură dată**: a doua utilizare aruncă `IllegalStateException`.

Pipeline: sursă → operații intermediare → operație terminală

```
// cu for
List<String> r = new ArrayList<>();
for (String c : cuvinte)
    if (c.startsWith("a"))
        r.add(c.toUpperCase());
r.sort(null);
```

```
// cu stream: CE vrem, nu CUM parcurgem
List<String> r = cuvinte.stream()
    .filter(c -> c.startsWith("a"))
    .map(String::toUpperCase)
    .sorted()
    .toList();
```

Crearea unui stream

`lista.stream()` · `Stream.of(1, 2, 3)` · `Arrays.stream(tablou)` · `Files.lines(path)` · `IntStream.range(0, n)` · pentru un `Iterable`: `StreamSupport.stream(it.spliterator(), false)`

Operații intermediare (întorc Stream)

`filter(Predicate)`	păstrează elementele care satisfac condiția
`map(Function)`	transformă fiecare element
`flatMap(Function)`	aplatizează: Stream<Stream<T>> → Stream<T>
`sorted()`, `sorted(Comparator)`	ordonează
`distinct()`	elimină duplicatele (equals)
`limit(n)`, `skip(n)`	primele n / sare peste n
`mapToInt(...)`	IntStream: sum, average

Operații terminale (produc rezultatul)

`forEach(Consumer)`	acțiune pe fiecare element
`toList()`, `collect(...)`	construiesc o colecție
`count()`	numărul de elemente (long)
`anyMatch / allMatch / noneMatch`	true / false
`reduce(identitate, op)`	combină elementele două câte două
`max / min(Comparator)`	`Optional<T>`
`findFirst()`	`Optional<T>`

Exemple

```
long promovati = studenti.stream().filter(s -> s.getMedia() >= 5).count();
boolean restante = studenti.stream().anyMatch(s -> s.getMedia() < 5);
double suma      = studenti.stream().map(Student::getMedia).reduce(0.0, Double::sum);
Optional<Student> best = studenti.stream().max(Comparator.comparing(Student::getMedia));
best.ifPresent(System.out::println);
```

Collectors

```
collect(Collectors.toSet())                                    // Set<T>
collect(Collectors.joining(", "))                              // "Ana, Dan, Barbu"
collect(Collectors.groupingBy(Student::getNume, Collectors.counting()))   // Map<String, Long>
```

Optional (rezultat care poate lipsi)

`ifPresent(c)` · `orElse(v)` · `orElseGet(supplier)` · `map(f)` · `orElseThrow()`. Nu apelăm `get()` fără verificare. Folosim `Optional` ca tip de retur, nu pentru câmpuri sau parametri.

> Funcțiile transmise lui `filter` și `map` nu au efecte secundare (nu modifică date din afara stream-ului). Rezultatul se construiește cu `collect` / `toList`, nu adăugând elemente într-o listă din `forEach`.